Adventure Works Cycle Analytics

This capstone project performs end-to-end business analytics for **Adventure Works Cycles**, a simulated global bicycle and accessories manufacturing company. The project combined SQL analytics, Power BI dashboarding, and machine learning modeling to uncover strategic business insights from multi-year transactional and customer data.

SQL | POWER BI | MACHINE LEARNING | 2015–2017 DATA

Presented By:

Prakash Chawda

Project Overview

The Adventure Works Analytics capstone was designed to transform raw relational data into actionable insights for leadership teams through structured analysis, visualization, and predictive intelligence. The dataset included historical sales records from 2015–2017, customer information, product catalog data, returns data, sales territories, and product categories. The project demonstrated how modern analytics ecosystems integrate database technologies, business intelligence platforms, and machine learning pipelines to support data-driven decision-making.

Layer 1: SQL Analytics

Business intelligence analysis for revenue, customer, product, return, and territory insights using structured querying and aggregation.

Layer 2: Power BI Dashboarding

Interactive executive storytelling and KPI tracking through a professional 5-page dashboard for leadership-level monitoring.

Layer 3: Machine Learning

Python-based models for forecasting revenue, predicting customer churn, and identifying high return-risk products.

The Problem

Adventure Works Cycles lacked unified visibility into its business operations across **revenue performance**, **customer purchasing behavior**, **product-level return risk**, and **future sales forecasting**. Business decisions were largely reactive due to fragmented reporting systems and limited predictive analytics capabilities. As the business expanded globally, the absence of integrated analytics increased the risk of inefficient inventory management, customer attrition, and delayed strategic decisions.

Product Performance Blind Spots

Inability to identify high-performing and underperforming products efficiently, making inventory and procurement decisions reactive rather than strategic.

Territory Profitability Gaps

Lack of visibility into territory-level profitability prevented targeted investment and resource allocation across geographic regions.

Customer Churn Risk

Limited understanding of repeat customer behavior and churn risks left the business vulnerable to silent attrition without early warning signals.

No Centralized Monitoring

No centralized dashboard for leadership-level monitoring meant executives lacked a single source of truth for operational KPIs.

Absence of Predictive Forecasting

No predictive forecasting models for revenue planning, making budget and growth projections largely guesswork.

Return Risk Identification

Difficulty identifying products with elevated return probability led to unexpected losses and inventory complications.

Objectives

The primary objectives of this project spanned three analytical domains — descriptive analytics, business intelligence visualization, and predictive machine learning — to generate data-driven business recommendations for operational improvement.

Descriptive & BI Objectives

- Analyze multi-year sales performance and identify revenue growth trends
- Evaluate category-level and product-level profitability
- Segment customers based on purchasing behavior and spending patterns
- Identify geographic regions contributing the highest sales and margins
- Analyze product return patterns and high-risk products
- Develop executive-level dashboards for business monitoring

Predictive & ML Objectives

- Forecast future revenue using machine learning techniques
- Predict customer churn using customer behavioral data
- Classify products with high return probability
- Generate data-driven business recommendations for operational improvement

All objectives were addressed through the integrated use of SQL, Power BI, and Python machine learning pipelines.

Dataset Overview

Dataset Tables Used

Sales	Transactional sales data from 2015–2017
Customers	Customer demographic and behavioral information
Products	Product catalog and pricing details
Returns	Product return records
Territories	Geographic sales territories
Product Categories	High-level product categorization
Product Subcategories	Detailed product classification

Key Data Domains

- Sales Revenue
- Order Quantity
- Product Pricing
- Profit Margin
- Customer Demographics
- Return Quantities
- Geographic Territories
- Product Categories
- Time-Series Sales Trends

Python for Data Science

Python was used for data cleaning, feature engineering, visualization, and predictive modeling. Python is the preferred ecosystem for machine learning pipelines because it provides extensive ML libraries, scalable preprocessing workflows, advanced modeling flexibility, statistical analysis capabilities, automation support, and production deployment readiness.

Libraries Used

Pandas	Data manipulation and preprocessing
NumPy	Numerical computation
Matplotlib	Data visualization
Scikit-learn	Machine learning models and evaluation
XGBoost	Advanced gradient boosting forecasting

Why Python?

Python's ecosystem is uniquely suited for end-to-end analytics workflows — from raw data ingestion through model training to result visualization. Its flexibility supports both exploratory analysis and production-grade model pipelines, making it ideal for a capstone project spanning multiple analytical domains.

Transfer Of Data To MySQL(A)

```python
#Import Libraries
import numpy as np
import pandas as pd
from sqlalchemy import create_engine

#Creating Engine
engine = create_engine("mysql+pymysql://root:1234@localhost:3306/adv_cycle",
                       pool_size=10 ,
                       max_overflow=20 , future=True)

#Load Dataset
customers = pd.read_csv(r"F:\project\Adventure\raw files\Customers.csv" , encoding = 'cp1252')
products  = pd.read_csv(r"F:\project\Adventure\raw files\Products.csv" , encoding = 'cp1252')
product_categories = pd.read_csv(r"F:\project\Adventure\raw files\Product_Categories.csv" , encoding = 'cp1252')
product_subcategories = pd.read_csv(r"F:\project\Adventure\raw files\Product_Subcategories.csv" , encoding = 'cp1252')
returns = pd.read_csv(r"F:\project\Adventure\raw files\Returns.csv" , encoding = 'cp1252')
sales_2015 = pd.read_csv(r"F:\project\Adventure\raw files\Sales_2015.csv" , encoding = 'cp1252')
sales_2016 = pd.read_csv(r"F:\project\Adventure\raw files\Sales_2016.csv" , encoding = 'cp1252')
sales_2017 = pd.read_csv(r"F:\project\Adventure\raw files\Sales_2017.csv" , encoding = 'cp1252')
territories = pd.read_csv(r"F:\project\Adventure\raw files\Territories.csv" , encoding = 'cp1252')
```

Transfer Of Data To MySQL(B)

```
#Transfer Data To MySQL
table_list = [(customers , "customers") , (products , "products") , (product_categories , "product_categories") ,
              (product_subcategories , "product_subcategories") , (returns , "returns") ,
              (sales , "sales") , (territories , "territories")]

for table,name in table_list:
    table.to_sql(name = name ,
                 con= engine , schema= "adv_cycle" ,
                 if_exists= "replace" , index= False,
                 method= "multi")


#Data Cleaning Process
sales = pd.concat([sales_2015 , sales_2016 , sales_2017] , ignore_index= True)

customers['BirthDate'] = pd.to_datetime(customers['BirthDate'],errors = "coerce")
customers['AnnualIncome'] = customers['AnnualIncome'].str.replace('[\$,]', '' , regex = True).astype(float)
customers['FullName'] = customers[['FirstName' , 'LastName']].agg(' '.join , axis = 1)
pos = customers.columns.get_loc('FirstName')
customers.drop(['FirstName' , 'LastName'], axis = 1 , inplace = True)
col = customers.pop('FullName')
customers.insert(pos , 'FullName' , col)

returns['ReturnDate'] = pd.to_datetime(returns['ReturnDate'] , errors = "coerce")

sales['OrderDate'] = pd.to_datetime(sales['OrderDate'] , errors = "coerce")
sales['StockDate'] = pd.to_datetime(sales['StockDate'] , errors = "coerce")
```

MySQL Analytics

MySQL was used for structured querying, data aggregation, joins, Common Table Expressions (CTEs), subqueries, and window functions. SQL is the industry standard for working with structured relational data because it enables fast querying of large datasets, efficient table joins and relationships, aggregation and KPI calculations, data transformation and filtering, scalable analytical workflows, and high-performance business intelligence extraction.

Join Operations

INNER JOIN and **LEFT JOIN** were used to combine data across the seven relational tables, enabling unified analysis of sales, customers, products, and territories.

CTEs & Subqueries

Common Table Expressions (CTEs) and subqueries organized complex multi-step analytical logic into readable, maintainable query structures.

Aggregation & Ranking

GROUP BY aggregations, **ranking functions**, and **window functions** enabled KPI calculations, revenue summaries, and product-level performance comparisons.

Date & Logic Functions

Date functions and **CASE statements** supported time-series analysis, conditional categorization, and dynamic segment labeling for business reporting.

SQL

Annual Revenue Trends

```
with yearly as
        (select extract(year from OrderDate) as Year,
sum(p.ProductPrice) as total_revenue  from sales s join products p
using (productkey) group by Year),
growth as
       (select  year , total_revenue , lag(total_revenue) over (order
by year) as prev_year_sale from yearly),
yoy as
        (select year , total_revenue,prev_year_sale,
round(((total_revenue - prev_year_sale)/prev_year_sale)*100 , 2)
as "YOY_growth" from growth)
select year ,total_revenue,prev_year_sale, YOY_growth from yoy;
```

Output:

year	total_revenue	prev_year_sale	YOY_growth
2015	6404933.580299864	NULL	NULL
2016	9187824.188003115	6404933.580299864	43.45
2017	9009718.270003024	9187824.188003115	-1.94

Sales rose steadily from 2015-2017,peaking in 2016. This shows consistent growth driven by higher order volumes and customer expansion

SQL

Category Performance

```
with category as
        (select pc.ProductCategoryKey , pc.CategoryName ,
p.ProductCost,p.ProductPrice  from sales s join products p using
(ProductKey) join product_subcategories ps using
(productsubcategorykey) join product_categories pc using
(productcategorykey)),
aggregation as
        (select distinct Productcategorykey , categoryName ,
round(sum(productprice - productcost) over(order by
productcategorykey),2) as revenue_margin ,
round(sum(productprice) over(order by productcategorykey),2) as
total_revenue from category)
select CategoryName,total_revenue,revenue_margin,rank()
over(order by total_revenue) as revenue_wise_rnk from
aggregation;
```

Output:

CategoryName	total_revenue	revenue_margin	revenue_wise_rnk
Bikes	23642495.1	9726168.27	1
Clothing	23954241.12	9860710.4	2
Accessories	24602476.04	10268688.96	3

Bikes leads total sales. Accessories has the best profit margin, while Clothing maintain moderate performance.

SQL

Top And Underperforming products

```
With product_summary as
(select p.ProductKey , p.ProductName , sum(s.OrderQuantity) as
total_quantity, round(sum(productprice),2)as total_revenue  from
sales s join products p using (productkey)  group by p.ProductKey ,
p.ProductName),
ranked as
 (select productkey , productname , total_quantity,total_revenue,
rank() over(order by total_quantity desc) as best_qty,  rank()
over(order by total_quantity asc) as worst_qty,   rank() over(order
by total_revenue desc) as best_rev,  rank() over(order by
total_revenue asc) as worst_rev            from product_summary)
select productkey ,productname , total_quantity , total_revenue
from ranked where best_qty = 1 or  worst_qty = 1 or best_rev = 1
or  worst_rev = 1;
```

Output:

productkey	productname	total_quantity	total_revenue
482	Racing Socks, L	509	2337.4
477	Water Bottle - 30 oz.	7967	19875.17
347	Mountain-100 Silver, 48	22	74799.78
362	Mountain-200 Black, 46	606	1241753.51

Top products Mountain-100 Silver , Mountain-200 Black. Underperforming products have low demand and higher returns.

SQL

Customers Segmented By Spend

```
with customer_spend as
(select s.customerkey , p.ProductKey , p.ProductName ,
sum(p.ProductPrice) as total_spend  from sales s join products p
using (productkey)  group by s.customerkey , p.ProductKey ,
p.ProductName),
segmented as
 (select * , ntile(3) over (order by total_spend) as segment_group
from customer_spend),
final as
 (select   case  when segment_group = 1 then "Low"
                 when segment_group = 2 then "Mid"
                   else "High"                    end as segment ,
round(sum(total_spend),2) as segmented_revenue from
segmented   group by segment_group)
select segment , segmented_revenue , round(segmented_revenue
* 100 / sum(segmented_revenue) over() , 2) as rev_share_percent
from final      order by segmented_revenue desc;
```

Output:

segment	segmented_revenue	rev_share_percent
High	23954900.84	97.37
Mid	542469.03	2.2
Low	105106.17	0.43

High Segmented customers have large part in the revenue contribution.

SQL

Geographic Sales Analysis

```
SELECT
    t.Country,    t.Region,    COUNT(DISTINCT s.OrderNumber) AS
total_orders,    SUM(s.OrderQuantity) AS total_quantity,
ROUND(SUM(p.productprice * s.orderQuantity) / COUNT(DISTINCT
s.ordernumber), 2) AS avg_order_value,
ROUND(SUM(p.ProductPrice), 2) AS total_sale
FROM    sales s        JOIN    territories t ON s.TerritoryKey =
t.SalesTerritoryKey
      JOIN    products p ON p.ProductKey = s.ProductKeyGROUP BY
t.Country , t.Region;
```

Output:

Country	Region	total_orders	total_quantity	avg_order_value	total_sale
Australia	Australia	6060	17951	1223.84	7355515.93
Canada	Canada	3024	10894	585.07	1724106.65
France	France	2315	7862	1020.58	2333460.99
Germany	Germany	2294	7950	1100.56	2495340.11
United Kingdom	United Kingdom	2771	9694	1047.48	2866962.24
United States	Central	9	30	349.23	2996.25
United States	Northeast	10	40	640.16	6280.91
United States	Northwest	3675	12513	842.2	3047803.02
United States	Southeast	14	49	827.54	11454.24
United States	Southwest	4992	17191	966.1	4758555.7

Australia and United States lead in total sales. Emerging regions indicate future potential.

SQL

Profitability By Territory

```
with territory_profit as
                ( select  t.Region , t.Country ,
round(sum(p.ProductCost * s.OrderQuantity),2) as total_cost,
round(sum(p.ProductPrice * s.OrderQuantity),2) as total_revenue,
round(sum((p.ProductPrice-p.ProductCost)* s.OrderQuantity),2) as
profit,                round((sum((p.ProductPrice-p.ProductCost)*
s.OrderQuantity) *100) /                    (sum(p.ProductPrice *
s.OrderQuantity)),2) as profit_margin_percent     from sales s
                join territories t on t.SalesTerritoryKey = s.TerritoryKey
join products p on p.ProductKey = s.ProductKey  group by t.Region
, t.Country)
select * , rank() over (order by profit_margin_percent desc) as
margin_rnkfrom territory_profit;
```

Output:

Region	Country	total_cost	total_revenue	profit	profit_margin_percent	margin_rnk
Central	United States	1705.62	3143.06	1437.44	45.73	1
Northeast	United States	3526.99	6401.57	2874.58	44.9	2
Southeast	United States	6452.33	11585.62	5133.29	44.31	3
Canada	Canada	1011401.62	1769245.81	757844.2	42.83	4
Northwest	United States	1780323.2	3095074.47	1314751.27	42.48	5
Southwest	United States	2782449.53	4822794.7	2040345.17	42.31	6
France	France	1373297.26	2362643.32	989346.06	41.87	7
United Kingdom	United Kingdom	1687787.68	2902562.09	1214774.42	41.85	8
Germany	Germany	1470493.77	2524679.97	1054186.21	41.76	9
Australia	Australia	4339433.39	7416456.2	3077022.81	41.49	10

U.S and Canada show highest profit margin. United Kingdom , Germany and Australia have lower margin due to operational costs.

SQL

Return Analysis

```
with sales_data as
        (select p.productkey , p.productname
,ps.Subcategoryname,pc.categoryname, sum(orderquantity) as
sale_quantity from sales s
        join products p using (productkey)
        join product_subcategories ps using
(productsubcategorykey)        join product_categories pc using
(productcategorykey)
group by productkey , productname ,
ps.Subcategoryname,pc.categoryname),
return_data as
        (select p.productkey ,p.productname , sum(returnquantity)
as return_quantity from returns r join products p using
(productkey)        group by productkey , productname)
select sd.productkey , sd.productname ,
sd.categoryname,sd.Subcategoryname,
coalesce(rd.return_quantity , 0) as total_returned,
round(coalesce(rd.return_quantity , 0)*100/(sale_quantity),2) as
return_percent from sales_data sd  left join return_data rd on
sd.productkey = rd.productkey;
```

Output:

productkey	productname	categoryname	Subcategoryname	total_returned	return_percent
332	Road-650 Black, 58	Bikes	Road Bikes	2	3.13
312	Road-150 Red, 48	Bikes	Road Bikes	8	4.47
350	Mountain-100 Black, 44	Bikes	Mountain Bikes	2	6.45
338	Road-650 Black, 44	Bikes	Road Bikes	0	0.00
310	Road-150 Red, 62	Bikes	Road Bikes	4	2.37
314	Road-150 Red, 56	Bikes	Road Bikes	5	3.18
345	Mountain-100 Silver, 42	Bikes	Mountain Bikes	0	0.00
313	Road-150 Red, 52	Bikes	Road Bikes	5	2.98
351	Mountain-100 Black, 48	Bikes	Mountain Bikes	2	5.56

Average return rate = X%. Bikes have slightly higher returns, especially Road-150 Red , 48. Suggests potential for quality review.

SQL

Sales Seasonality

```
with monthly_sales as
(select date_format(orderdate , '%Y-%m') as month ,
round(sum(OrderQuantity*productprice),2) as total_revenue
from sales s join products p using (productkey)  group by month),
trend as
(select month , total_revenue , lag(total_revenue) over
(order by month) as prev_month,  lead(total_revenue) over(order
by month) as next_month   from monthly_sales) select month ,
total_revenue,      case      when total_revenue > prev_month and
total_revenue > next_month then "peak" when total_revenue <
prev_month and total_revenue < next_month then "Trough"
else "Normal"  end as trend_typefrom trend;
SELECT    CONCAT(YEAR(orderdate),          '-Q',
QUARTER(orderdate)) AS Quarter,    ROUND(SUM(OrderQuantity *
productprice), 2) AS total_revenueFROM    sales s       JOIN
products p USING (productkey)GROUP BY QuarterORDER BY
Quarter;
```

Output:

month	total_revenue	trend_type
2015-01	585312.65	Normal
2015-02	532226.25	Trough
2015-03	643436.1	Normal
2015-04	653364.04	Normal
2015-05	659325.9	Normal
2015-06	669988.67	peak
2015-07	486115.01	Trough

Quarter	total_revenue
2015-Q1	1760975
2015-Q2	1982678.6
2015-Q3	1366630.7
2015-Q4	1294649.28
2016-Q1	1378550.4
2016-Q2	1574317.14
2016-Q3	2572293.35
2016-Q4	3799042.9
2017-Q1	4062216.08
2017-Q2	5123233.36

Sales peak in 2017-Q2 , particularly June , aligning with holiday and promotional cycles.

SQL

New V/s Repeat Customers

```
with first_purchase as
        (select s.CustomerKey,min(s.OrderDate) over() as
first_order_date ,p.ProductPrice  from sales s   join products p
using (productkey)),
classified as
        (select f.* ,s.OrderDate,
         case  when s.OrderDate = f.first_order_date then "New"
else "Repeat“
         end as customer_type
                from sales s
       join first_purchase f on s.customerkey = f.customerkey),
monthly_data as
        (select date_format(orderdate , '%Y-%m') as
month,customer_type , round(sum(productprice),2) as
total_revenue          from classified  group by month ,
customer_type)
select month , customer_type ,
total_revenue,round((total_revenue * 100) / (sum(total_revenue)
over (partition by month)),2) as revenue_share_percentfrom
monthly_data order by month, customer_type;
```

Output:

month	customer_type	total_revenue	revenue_share_percent
2015-01	New	15807.94	1.74
2015-01	Repeat	890526.19	98.26
2015-02	Repeat	793625.94	100
2015-03	Repeat	1098664.46	100
2015-04	Repeat	1083967.33	100
2015-05	Repeat	1104238.73	100
2015-06	Repeat	1034735.89	100
2015-07	Repeat	973660.98	100
2015-08	Repeat	985093.82	100
2015-09	Repeat	733674.53	100

Returning customers contribute high part in revenue , showing strong loyalty. New customers increased showing effective acquisition.

Power BI Dashboarding

Power BI was used to build a professional interactive **5-page dashboard** for executive storytelling and KPI tracking. The Power BI enables leadership-level storytelling because it combines interactive visualizations, real-time KPI monitoring, dynamic filtering, drill-through analysis, executive dashboard design, and cross-functional business storytelling.

1 **Revenue & Growth Analysis**
Year-over-year revenue trends, growth rates, and margin performance visualized for executive monitoring.

2 **Product & Category Insights**
Category-level and product-level profitability breakdowns with drill-through capabilities for detailed exploration.

3 **Customer Analysis**
Customer segmentation, purchasing behavior patterns, and spending distribution across customer groups.

4 **Regional Performance**
Geographic territory analysis showing sales contribution, margins, and regional growth comparisons.

5 **Seasonality Analysis**
Time-series patterns revealing seasonal demand fluctuations and peak sales periods across product categories.

CYCLE ADVENTURES INDIA

Revenue & Growth

Category

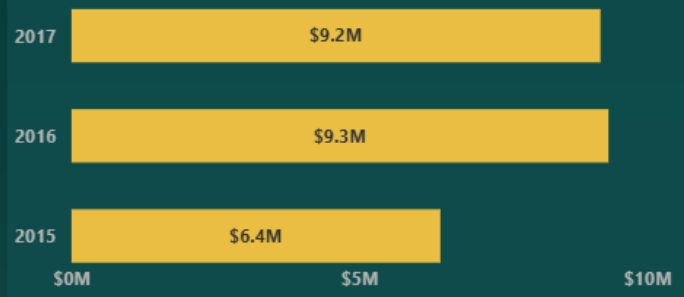
- Accessories
- Bikes
- Clothing
- Components

Continent

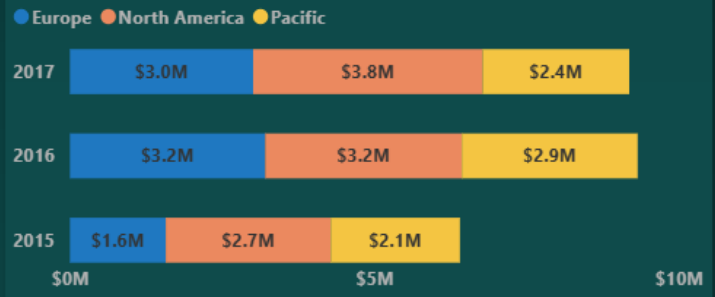
- Europe
- North America
- Pacific

$24.91M	25K	58.40%	$990
Total Revenue	Total Orders	YOY Growth(%)	Avg Order Value

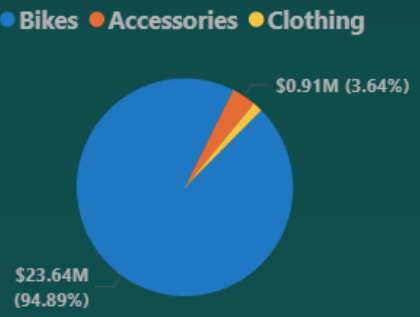
Year Wise Revenue



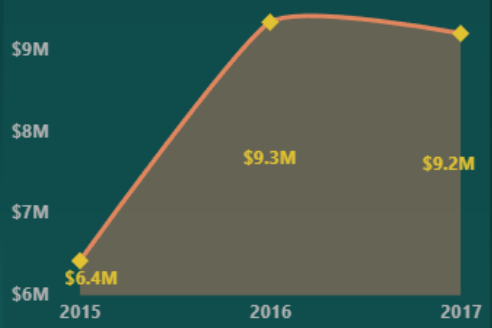
Year Based Continents Revenue



Category Wise Revenue



Total Revenue by Year



Quarter	Total Revenue	YOY Growth(%)
Qtr 2	$86,80,229.11	144.03%
Qtr 1	$72,01,741.48	129.39%
Qtr 4	$50,93,692.18	0.00%
Qtr 3	$39,38,924.05	0.00%

Dashboard Insights

Revenue & Growth

- Revenue shows a consistent upward trajectory, with notable growth in the final year.
- Year over Year growth rate remained positive across all periods, reflecting strong business expansion .
- Average Order value increased slightly year-over-year, suggesting higher-value purchases or better cross-selling.

Products & Category Insights

- Bikes remain the top-performing category, accounting for nearly of total revenue.
- Clothing and Accessories categories provide strong profit margin despite moderate revenue shares.
- A few high – revenue products also show above-average return rates, signaling potential products quality or expectation gaps
- Margin band analysis highlights that mid-range products offer the most balanced performance between sales volume and profitability.

Product & Category Insights

84K	1828	41.97%	2.17%
Total Quantity	Total Returns	Profit Margin(%)	Return Rate(%)

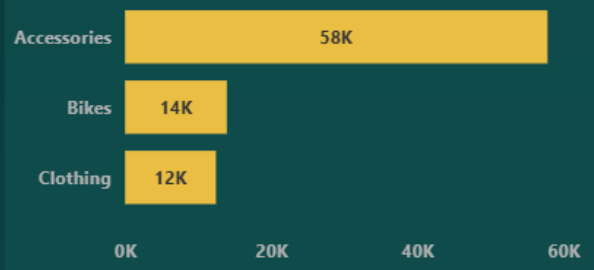
Continent

- [ ] Europe
- [ ] North America
- [ ] Pacific

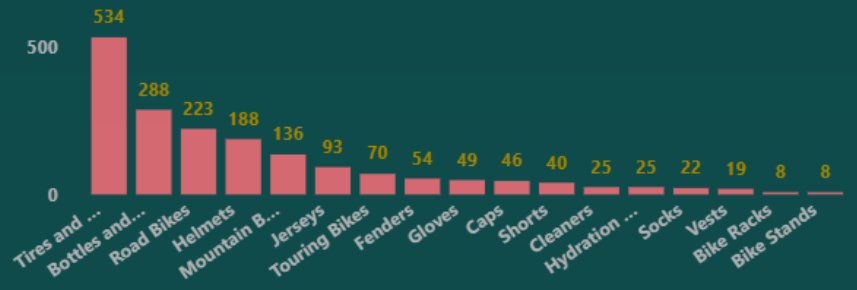
Gender

- [ ] Female
- [ ] Male
- [ ] NA

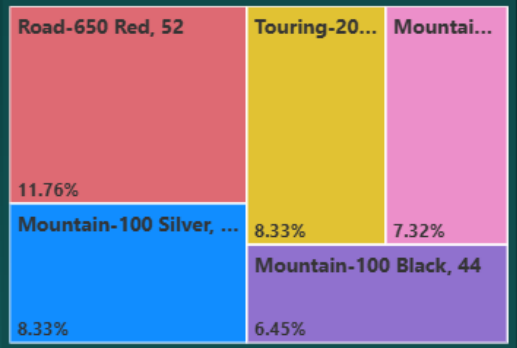
Category Wise Total Quantity Orders



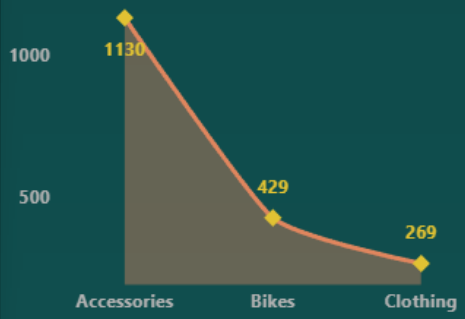
Subcategory Wise Total Returns



Products wise Return Rate



Category Wise Return Quantity



CategoryName	Total Returns	Return Rate(%)
Accessories	1130	1.95%
Bikes	429	3.08%
Clothing	269	2.16%

CYCLE ADVENTURES INDIA

Customer Analysis

Category

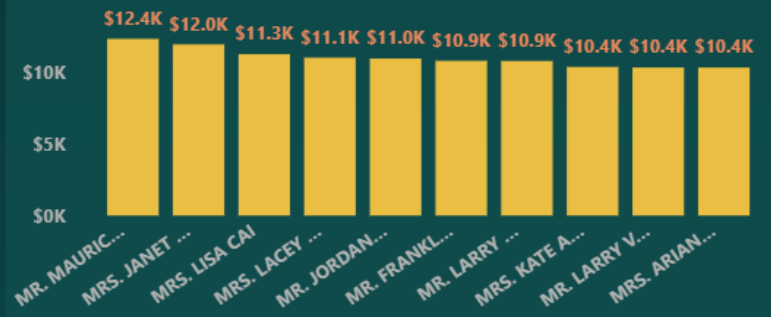
- Accessories
- Bikes
- Clothing
- Components

Continent

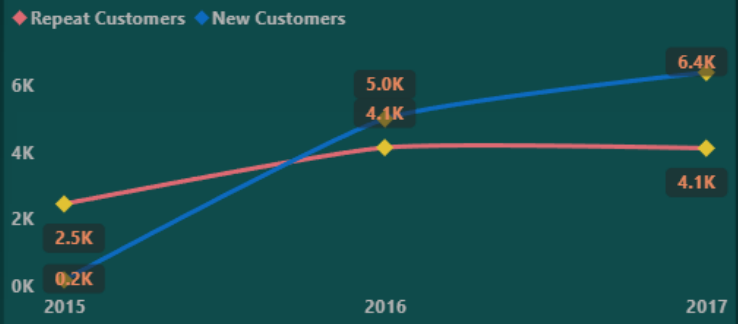
- Europe
- North America
- Pacific

17K	5857	33.63%	1.43K
Total Customers	Repeat Customers	Repeat Purchase Rate(%)	Avg Spend Per Customers

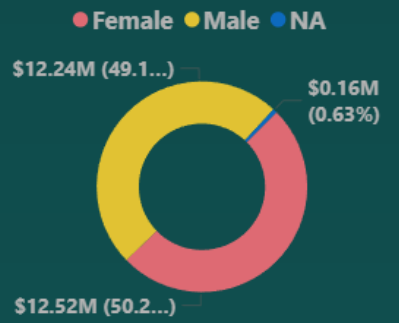
Top 10 Customers By Revenue



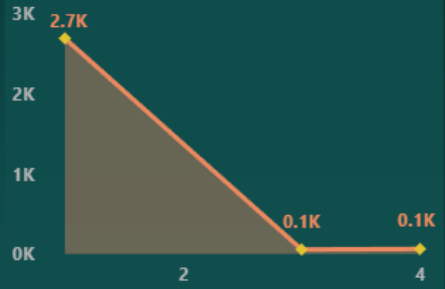
New Vs Repeat Customers Over Time



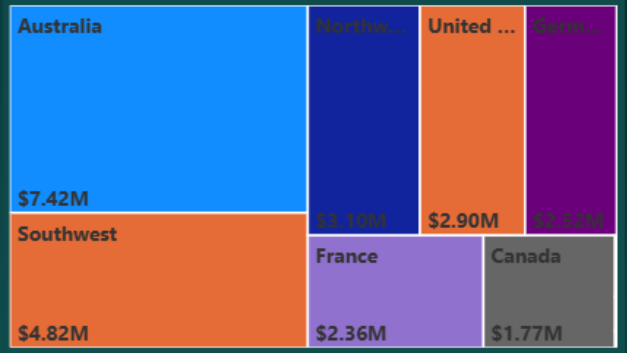
Gender Wise Total Revenue



Avg Spend Per Customers by ProductCategoryKey



Region Wise Sales



Dashboard Insights

Customers Analysis

- Customers base continues to expand with a healthy mix of new and repeat purchase.
- Repeat customers generate the majority of sales , showing strong loyalty and retention performance.
- Average spend distribution is skewed toward mid and high value buyers , suggesting successful upselling and premium positioning.

Products & Category Insights

- U.S and Australia dominate total revenue , driven by large customer bases and higher AOVs.
- Pacific territories show high profit margins despite smaller sales volumes , highlighting operational efficiency.
- Emerging markets show year-over-year growth potential nut require targeted expansion efforts.
- Territory ranking highlights strong performance consistency in Northwest and Central regions.

Seasonality

- Revenue reached its highest performance in 2016, indicating that the business achieved peak sales growth during that year.
- June 2017 was the highest revenue-generating month, showing strong mid-year seasonal demand for products.
- The United States contributed the highest revenue among all countries , while Canada generated the lowest revenue, highlighting a significant regional performance gap..

CYCLE ADVENTURES INDIA

$24.91M	25K	42.95%	10.46M
Total Revenue	Total Orders	Avg Profit Margin By Territory	Profit

Regional Performance

Category

- [ ] Accessories
- [ ] Bikes
- [ ] Clothing
- [ ] Components

Year & Month

- [ ] 2015
- [ ] 2016
- [ ] 2017

Country Wise Revenue

Region Wise Revenue

Country/Region Wise Revenue

Europe North America Pacific

Region	Total Revenue	Territory Rank
Australia	$74,16,456.20	1
Southwest	$48,22,794.70	2
Northwest	$30,95,074.47	3
United Kingdom	$29,02,562.09	4
Germany	$25,24,679.97	5
France	$23,62,643.32	6
Canada	$17,69,245.81	7
Southeast	$11,585.62	8
Northeast	$6,401.57	9
Central	$3,143.06	10

CYCLE ADVENTURES INDIA

Seasonality

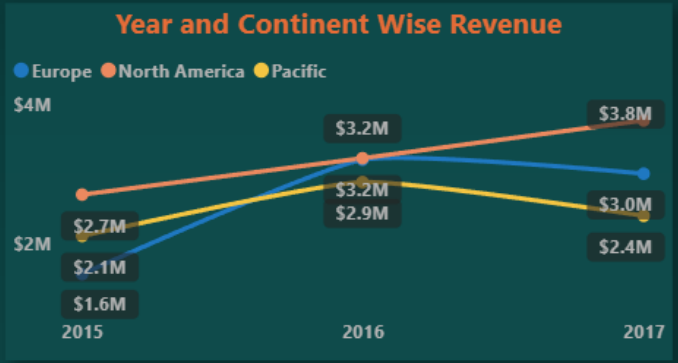
Category & Subcategory

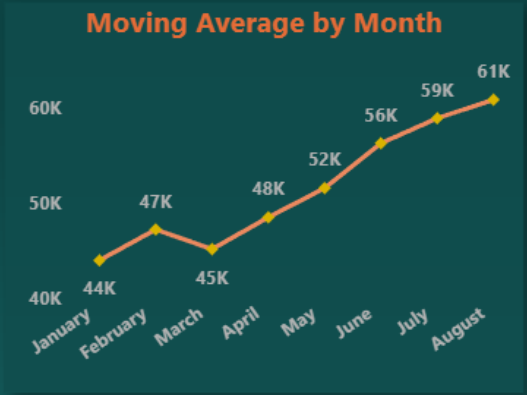
- Accessories
- Bikes
- Clothing
- Components

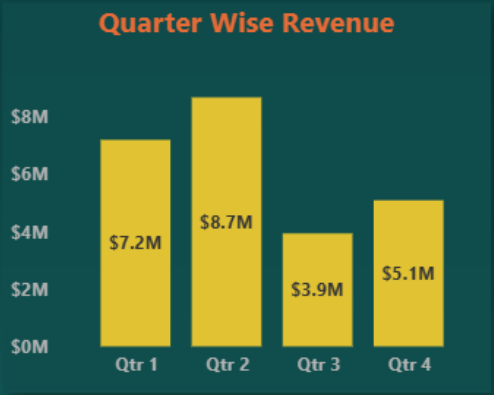
Continent & Country

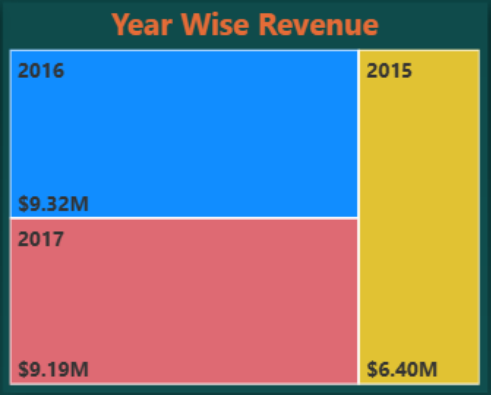
- Europe
- North Ame...
- Pacific

2016	2017-Jun	United States	Canada
Peak Year	Peak Month Year	Top Country	Bottom Country









Machine Learning Models

Three ensemble machine learning models were developed in Python to address the predictive objectives of the project. Ensemble methods were selected instead of simpler models because they improve predictive accuracy, reduce overfitting risk, handle nonlinear relationships effectively, capture complex business patterns, provide robust performance on structured business datasets, and work effectively with mixed feature types. These models were particularly suitable for customer behavior prediction and revenue forecasting tasks.

XGBRegressor

Purpose: Revenue Forecasting

XGBoost's gradient boosting regressor was used to forecast future revenue by learning from historical sales patterns, seasonal trends, and product-level signals in the time-series data.

RandomForestClassifier

Purpose: Customer Churn Prediction

A Random Forest classifier predicted which customers were at risk of churning based on purchasing behavior, frequency, recency, and demographic features.

GradientBoostingClassifier

Purpose: Return Risk Classification

A Gradient Boosting classifier identified products with high return probability, enabling proactive inventory and quality management decisions.

Revenue Forecasting Model(A)

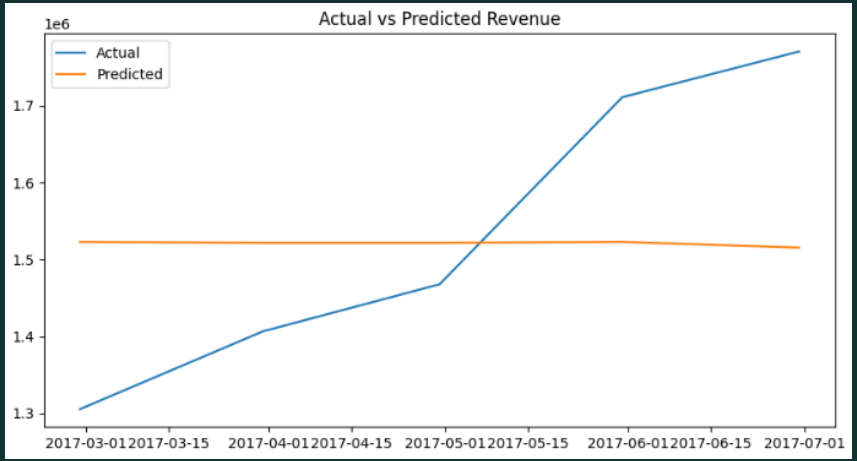
```python
#Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split


#merge Data
df = sales.merge(products[['ProductKey' , 'ProductPrice']] , on = 'ProductKey' , how = 'left')
df['Revenue'] = df['OrderQuantity'] * df['ProductPrice']
returns_df = returns.merge(products[['ProductKey', 'ProductPrice']], on = 'ProductKey' , how = 'left')
returns_df['Return_value'] = returns_df['ReturnQuantity'] * returns_df['ProductPrice']

#Aggregation
sales_agg = df.groupby( by = 'OrderDate' ).agg(func= {'Revenue' : 'sum'}).reset_index()
returns_agg = returns_df.groupby(by = 'ReturnDate' ).agg(func = {'Return_value' : 'sum'}).reset_index()

#Combine Sales & Returns
final_df = pd.merge(sales_agg , returns_agg , left_on = 'OrderDate',
                    right_on = 'ReturnDate' , how = 'left')
final_df['Return_value'] = final_df['Return_value'].fillna(0)
final_df['Net_revenue'] = round(final_df['Revenue'] - final_df['Return_value'],2)

#Resample to monthly data
final_df.set_index('OrderDate' , inplace = True)
monthly = final_df['Net_revenue'].resample('ME').sum()

#Convert time series
df_ml = monthly.to_frame(name = 'Revenue')

#Creating Lag Features
for lag in range(1,7):
    df_ml[f'lag_{lag}'] = df_ml['Revenue'].shift(lag)

df_ml['time_index'] = np.arange(len(df_ml))
```

Revenue Forecasting Model(B)

```
#Creating Rolling Features
df_ml['rolling_mean_3'] = df_ml['Revenue'].rolling(3).mean()
df_ml['rolling_std_3'] = df_ml['Revenue'].rolling(3).std()

#Add time based features
df_ml['month'] = df_ml.index.month
df_ml['Quarter'] = df_ml.index.quarter
df_ml['Year'] = df_ml.index.year

#Drop Missing Values
df_ml.dropna(inplace = True)

#Split Features & Target
split = int(len(df_ml)*0.8)
train = df_ml[:split]
test = df_ml[split : ]

#Train Test Split
x_train = train.drop('Revenue' , axis = 1)
y_train = train['Revenue']
x_test = test.drop('Revenue' , axis = 1)
y_test = test['Revenue']

#Train xgboost (XGBRegressor)
from xgboost import XGBRegressor
model = XGBRegressor(n_estimators = 500 , learning_rate = 0.03 , max_depth = 5 , subsample = 0.8,
                     colsample_bytree = 0.8 , random_state = 42)
model.fit(x_train , y_train)
```

Revenue Forecasting Model(C)

```python
#Predict
y_pred = model.predict(x_test)

#Evaluate Model
mae = mean_absolute_error(y_test , y_pred)
mse = mean_squared_error(y_test , y_pred)
rmse = np.sqrt(mse)
print("MAE : " , mae)
print("RMSE : " , rmse)

#Plot Between Actual V/S Predicted
plt.figure(figsize = (10,5))
plt.plot(y_test.index, y_test , label = 'Actual')
plt.plot(y_test.index , y_pred , label = 'Predicted')
plt.legend()
plt.title("Actual vs Predicted Revenue")
plt.show()

#Forecast Next 3 months
last_row = df_ml.drop('Revenue' , axis = 1).iloc[-1:].copy()
future_preds = []
for i in range(3):
    pred = model.predict(last_row)[0]
    future_preds.append(pred)

    for lag in range(6,1,-1):
        last_row[f'lag_{lag}'] = last_row[f'lag_{lag-1}']

    last_row['lag_1'] = pred

print("Next 3 months forecast : " , future_preds)
```



```python
#Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
from sklearn.metrics import accuracy_score
from datetime import datetime
```

Churn Prediction (Customer Retention) Model(A)

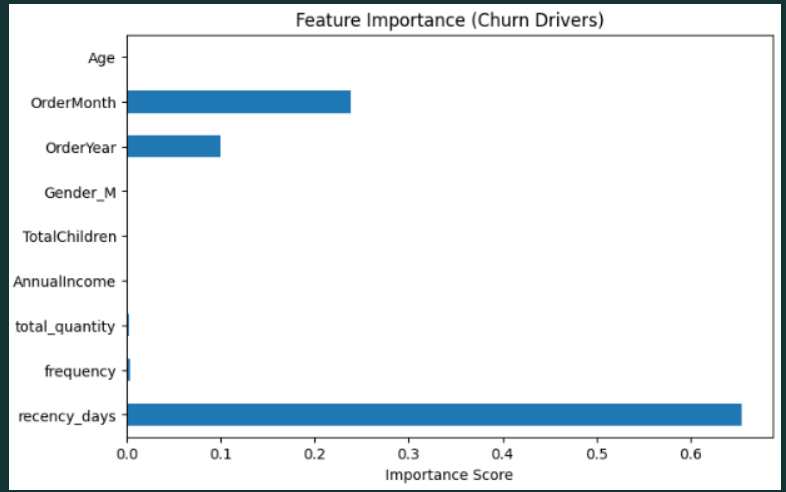
```python
#Create Customers Level Dataset
cp = sales.merge(customers , on = 'CustomerKey' , how = 'left')

#Features Engineering
latest_date = cp['OrderDate'].max()
recency = cp.groupby('CustomerKey')['OrderDate'].max().reset_index()
recency['recency_days'] = (latest_date - recency['OrderDate']).dt.days

#Numbers of Orders
frequency = cp.groupby('CustomerKey')['OrderNumber'].nunique().reset_index()
frequency.columns = ['CustomerKey' , 'frequency']

#Total Quantity Purchased
monetary = cp.groupby('CustomerKey')['OrderQuantity'].sum().reset_index()
monetary.columns = ['CustomerKey' , 'total_quantity']

#Merge Features
features = recency.merge(frequency , on = 'CustomerKey')
features = features.merge(monetary , on = 'CustomerKey')

#Create Target Variable(Churn Label)
features['churn'] = np.where(features['recency_days'] > 90 , 1, 0)

#Add Customers Demographics
final_cp = features.merge(customers , on = 'CustomerKey' , how = 'left')

#Drop Unnecessary columns
final_cp = final_cp.drop(columns = ['Prefix' , 'FullName' , 'MaritalStatus' , 'EmailAddress' ,
                                    'HomeOwner' , 'EducationLevel' , 'Occupation'])

#Encodes categorical variable
final_cp = pd.get_dummies(final_cp , drop_first = True)
```

Churn Prediction (Customer Retention) Model(B)

```
#Define p and q
p = final_cp.drop(columns = ['churn' , 'CustomerKey'])
q = final_cp['churn']

#Train Test Split
p_train , p_test , q_train , q_test = train_test_split(p,q,test_size = 0.2 , random_state = 42)

#Convert Date Columns
p_train['OrderYear'] = p_train['OrderDate'].dt.year
p_train['OrderMonth'] = p_train['OrderDate'].dt.month
p_test['OrderYear'] = p_train['OrderDate'].dt.year
p_test['OrderMonth'] = p_train['OrderDate'].dt.month

#Convert Birthdate to age
current_year = datetime.now().year
p_train['Age'] = current_year - p_train['BirthDate'].dt.year
p_test['Age'] = current_year - p_test['BirthDate'].dt.year

#Drop Original Date Columns
p_train = p_train.drop(columns = ['OrderDate' , 'BirthDate'])
p_test = p_test.drop(columns = ['OrderDate' , 'BirthDate'])

#Convert Boolean to integers
p_train['Gender_M'] = p_train['Gender_M'].astype(int)
p_test['Gender_M'] = p_test['Gender_M'].astype(int)
```

Churn Prediction (Customer Retention) Model(C)

```python
#Train Random Forest Model
rf = RandomForestClassifier(
    n_estimators = 200 , max_depth = 8 , random_state= 42
)
rf.fit(p_train , q_train)

#Prediction
q_pred = rf.predict(p_test)
results = pd.DataFrame({
    'CustomerKey' : p_test.index,
    'Actual' : q_test,
    'Predicted_churn' : q_pred
})
churn_customers = results[results['predicted_churn'] == 1]
print(churn_customers.head())

# Model Evaluation
print("Accuracy : " , accuracy_score(q_test , q_pred))
print(classification_report(q_test , q_pred))

#Feature Importance
importances = pd.Series(rf.feature_importances_,index = p_train.columns)
print(importances.sort_values(ascending = False).head(10))

#Plot for Churn drivers
plt.figure(figsize= (8,5))
importances.plot(kind = 'barh')
plt.title("Feature Importance (Churn Drivers)")
plt.xlabel("Importance Score")
plt.show()
```



High Return Risk Products(A)

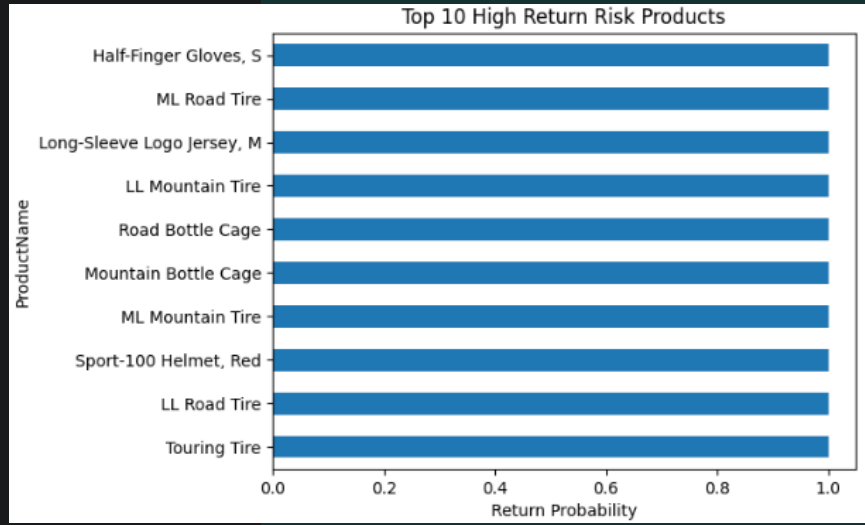
```python
# Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import classification_report , roc_auc_score
from sklearn.preprocessing import LabelEncoder

#Merge Dataset
hr = sales.merge(returns , on = ['ProductKey' , 'TerritoryKey'] , how = 'left')
hr = hr.merge(customers , on = 'CustomerKey' , how = 'left')
hr = hr.merge(products , on = 'ProductKey'  , how = 'left')
hr = hr.merge(territories , left_on = 'TerritoryKey' , right_on= 'SalesTerritoryKey' , how = 'left')

#Target Variable
hr['ReturnQuantity'] = hr['ReturnQuantity'].fillna(0)
hr['Returned'] = (hr['ReturnQuantity'] > 0 ).astype(int)

#Drop Useless Columns
drop_cols = ['FullName','EmailAddress' , 'ProductDescription' , 'ProductSKU' , 'OrderNumber']
hr.drop(columns = [col for col in drop_cols if col in hr.columns], inplace = True , errors = 'ignore')

#Handle Dates
date_cols = ['OrderDate' , 'StockDate' , 'ReturnDate' , 'BirthDate']
for col in date_cols:
    if col in hr.columns:
        hr[col] = pd.to_datetime(hr[col] , errors = 'coerce')

if 'BirthDate' in hr.columns:
    hr['Age'] = 2026 - hr['BirthDate'].dt.year

if 'OrderDate' in hr.columns and 'StockDate' in hr.columns:
    hr['OrderDelayDays'] = (hr['OrderDate'] - hr['StockDate']).dt.days

hr.drop(columns = [c for c in date_cols if c in hr.columns] , inplace = True , errors = 'ignore')
```

High Return Risk Products(B)

```python
#Feature Engineering
if 'ProductPrice' in hr.columns and 'ProductCost' in hr.columns:
    hr['Profit'] = hr['ProductPrice'] - hr['ProductCost']

if 'ProductKey' in hr.columns:
    hr['product_return_rate'] = hr.groupby('ProductKey')['Returned'].transform('mean')

#Drop Ids
hr.drop(columns = ['ProductKey' ,'CustomerKey' ] , inplace = True , errors = 'ignore')

#Handle missing values
num_cols = hr.select_dtypes(include = [np.number]).columns
hr[num_cols] = hr[num_cols].fillna(0)

cat_cols = hr.select_dtypes(include= 'object').columns
hr[cat_cols] = hr[cat_cols].fillna('Unknown')


product_names = hr['ProductName'].copy()

#Encoding (No Memory Issue)
le = LabelEncoder()
for col in hr.select_dtypes(include= 'object').columns:
    hr[col] = le.fit_transform(hr[col])

#Split Data
A = hr.drop('Returned' , axis = 1)
B = hr['Returned']

A_train , A_test , B_train , B_test = train_test_split(A,B,test_size = 0.2 , random_state = 42)
```

High Return Risk Products(C)

```python
#Train Model
model = GradientBoostingClassifier(
    n_estimators = 120 , learning_rate = 0.1 , max_depth = 5 , random_state = 42)
model.fit(A_train , B_train)
#Evaluation
B_pred = model.predict(A_test)
B_prob = model.predict_proba(A_test)[:,1]

print(classification_report(B_test , B_pred))
print("AUC : " , roc_auc_score(B_test , B_prob))

#High Return Products
hr['Return_Prob'] = model.predict_proba(A)[:,1]
threshold = 0.6
high_return_products = hr[hr['Return_Prob'] > threshold]
high_return_products = high_return_products[['ProductName','Return_Prob','ProductPrice','ProductCost' , 'Region']]
high_return_products = high_return_products.sort_values(by= 'Return_Prob' , ascending= False)
print(high_return_products.head(20))

#Features Importance
importance = pd.Series(model.feature_importances_ , index = A.columns)
print(importance.sort_values(ascending = False).head(10))

#High Return Risk Products
results = A_test.copy()
results['Return_Prob'] = B_prob
results['ProductName'] = product_names.loc[results.index]

high_risk_product_top10 = (results.groupby('ProductName')['Return_Prob']
                           .mean().sort_values(ascending = False).head(10))
high_risk_product_top10.plot(kind = 'barh')
plt.title("Top 10 High Return Risk Products")
plt.xlabel("Return Probability")
plt.ylabel("ProductName")
plt.gca().invert_yaxis()
plt.show()
```



Key Takeaways

The Adventure Works Analytics capstone demonstrated how modern analytics ecosystems integrate database technologies, business intelligence platforms, and machine learning pipelines to support data-driven decision-making. By combining SQL, Power BI, and Python, the project addressed all ten objectives — from multi-year sales analysis and customer segmentation to revenue forecasting, churn prediction, and return risk classification.

Integrated Analytics Architecture

The three-layer approach — SQL for data extraction, Power BI for visualization, and Python for prediction — showcases how each tool plays a distinct but complementary role in a complete analytics solution.

Ensemble ML for Business Problems

XGBoost, Random Forest, and Gradient Boosting provided robust, accurate predictions for revenue forecasting, customer churn, and return risk — proving ensemble methods are well-suited for structured business datasets.

From Reactive to Proactive Decision-Making

By addressing fragmented reporting, absent forecasting, and limited customer visibility, this project illustrates how integrated analytics transforms reactive operations into proactive, insight-driven strategy.

This capstone serves as a replicable blueprint for building end-to-end analytics solutions in real-world business environments using SQL, Power BI, and Python machine learning.

What We Accomplished

The project successfully delivered an end-to-end analytics implementation demonstrating practical expertise in data analytics, business intelligence, and predictive machine learning.

Analytics Framework

Centralized business analytics framework with SQL-based business intelligence analysis

Interactive Dashboards

Power BI executive dashboards with revenue trend monitoring systems

Business Analysis

Product profitability analysis, customer segmentation models, and territory profitability evaluation

Predictive Modeling

Revenue forecasting using XGBoost, customer churn prediction using Random Forest, and product return risk classification using Gradient Boosting

The project also delivered strategic business recommendations and end-to-end analytics implementation.

Limitations

Although the project achieved strong analytical results, several limitations were identified in model performance and data quality.

Underfitting Concern

The revenue forecasting model underfit rapid business growth trends, limiting its ability to capture accelerating patterns in the data.

Data Leakage Risk

The churn and return models achieved **100% accuracy**, indicating possible data leakage. Features such as `ReturnQuantity` were highly predictive and may have inflated performance metrics.

⚠ 100% accuracy in real-world classification tasks is a red flag — it typically signals that the model is learning from information it shouldn't have access to during training.

Limitations

Beyond model performance, several constraints related to data availability and analytical scope were identified.

Limited Time Horizon

The dataset covered only **2015–2017**, restricting the ability to capture longer-term trends and seasonal patterns.

Missing External Variables

External variables such as **marketing spend** and **economic indicators** were unavailable, limiting the models' contextual understanding of business drivers.

No Real-Time Analytics

Real-time analytics integration was not implemented, meaning insights are based on historical snapshots rather than live data streams.

No Sentiment Analysis

Customer sentiment analysis was not included, leaving a gap in understanding qualitative customer feedback and satisfaction drivers.

Future Opportunities: Infrastructure & Deployment

The project can be enhanced further through multiple advanced analytical improvements, beginning with infrastructure modernization.

1

Real-Time Dashboards

Integrate real-time dashboard connectivity for live business monitoring

2

Cloud Architecture

Implement cloud-based analytics architecture for scalability and flexibility

3

API Deployment

Deploy models as APIs to enable seamless integration with operational systems

Future Opportunities: Advanced Analytics

Further enhancements can deepen analytical capabilities and expand predictive intelligence across the business.

Model Retraining

Retrain models using more historical data to improve accuracy and reduce underfitting

Recommendation Systems

Build recommendation systems to personalize customer experiences and drive cross-sell

Anomaly Detection

Add anomaly detection systems to identify unusual patterns in real time

Customer Lifetime Value

Develop customer lifetime value prediction models to guide retention investment

Inventory Optimization

Build inventory optimization models to reduce waste and improve supply chain efficiency

Deep Learning Forecasting

Introduce deep learning forecasting models to capture complex, non-linear patterns

Future Opportunities: External Business Drivers

Enriching models with external data will significantly improve predictive accuracy and strategic relevance.

External Drivers to Integrate

- Marketing campaigns
- Holiday events
- Economic conditions

Why It Matters

The current models lack visibility into key external forces that shape revenue, churn, and product demand. Integrating marketing spend data, seasonal holiday calendars, and macroeconomic indicators will enable more accurate forecasting and richer customer segmentation. These external business drivers were identified as unavailable in the current dataset but are critical for next-generation model performance.

Strategic Recommendations

Five areas of strategic action are recommended to translate analytical insights into business impact.

Customer Retention

Focus retention campaigns on customers with `recency_days > 90`. Create personalized marketing campaigns and introduce loyalty reward programs.

Product Quality

Audit Tires & Tubes due to **534 returns**. Improve quality checks for high-return products and optimize inventory for low-performing products.

Revenue Growth

Increase Q2 marketing investment. Expand premium bicycle campaigns and focus on high-margin categories.

Regional Strategy

Prioritize Australia and United States territories. Expand operations in high-performing regions and investigate low-performing territories.

Predictive Analytics Recommendations & Closing

To sustain and improve model performance over time, the following practices are recommended for the predictive analytics function.

1. **Continuously Retrain Models**

   Regularly retrain machine learning models with fresh data to maintain accuracy as business conditions evolve

2. **Reduce Data Leakage**

   Audit and remove leakage-prone features in classification tasks to ensure realistic, deployable model performance

3. **Integrate Predictive Outputs**

   Embed predictive outputs directly into business planning systems to operationalize insights and drive decision-making

The project demonstrated practical expertise in data analytics, business intelligence, and predictive machine learning — providing a strong foundation for continued analytical advancement.

LinkedIn.com GitHub.com

Conclusion

The Adventure Works Analytics Capstone successfully delivered a comprehensive business intelligence and predictive analytics solution — transforming complex transactional data into meaningful insights capable of supporting strategic growth, operational efficiency, and future planning.

What Was Achieved

- Integrated SQL, Power BI, and Python-based ML into a unified solution
- Demonstrated technical analytical expertise across the full data lifecycle
- Emphasized practical business value in solving real-world challenges

Industry Alignment

The project represents a **professional enterprise analytics implementation** aligned with modern industry standards in data analytics, business intelligence, and machine learning — ready to support strategic, operational, and predictive decision-making.

This capstone Project available on the GitHub and LinkedIn . Provided Link are Clickable.

WSCube-Tech